

СОПРОВОДИТЕЛЬНАЯ ДОКУМЕНТАЦИЯ

ФСП Карьера

Платформа подбора ИТ-специалистов с обратной механикой:
категория по тесту, выход на кандидата по инициативе работодателя,
профиль с достижениями ФСП

Специальный трек ФСП · хакатон «Лидеры цифровой трансформации — 2026»

Версия 1.0 · 5 октября 2026 г.

359 семейств заданий с уникальными вариантами для каждого кандидата

0,845 доля релевантных в топ-10 подборки (фильтры по резюме — 0,743)

2,4% завышения грейда по тесту против 36,1% в самооценке резюме

86 методов API в OpenAPI, 491 автотест, запуск одной командой Docker

Содержание

1. Обзор решения	3
1.1. Задача и подход	3
1.2. Роли и сквозной сценарий	3
1.3. Соответствие требованиям ТЗ	4
2. Функциональная архитектура	6
2.1. Функциональные модули	6
2.2. Путь кандидата	6
2.3. Жизненный цикл приглашения и отклика	6
2.4. Правила видимости данных	7
3. Компонентная архитектура	8
3.1. Схема компонентов	8
3.2. Компоненты и ответственность	8
3.3. Модель данных	9
3.4. Производительность и масштабирование	9
4. Механика тестирования	10
4.1. Опрос и выбор уровня	10
4.2. Банк заданий: семейства и варианты	10
4.3. Модель IRT и оценка уровня	10
4.4. Адаптивный алгоритм	11
4.5. Решение по грейду и правила пересдачи	11
4.6. Обоснование устойчивости к распространению заданий	12
4.7. Профиль по доменам и статусы навыков	12
5. Механика категоризации и подбора	13
5.1. Категория кандидата	13
5.2. Разбор потребности работодателя (NLP)	13
5.3. Рекомендованные категории и аналитика рынка	13
5.4. Ранжирование	13
5.5. Сила профиля и достижения ФСП	14
5.6. Объяснения и вероятность отклика	14
5.7. Подборки, фильтры и поиск по банку	14
5.8. Регулярные задания работодателей	15
6. Процедура валидации и результаты	16
6.1. Принцип	16
6.2. Тестирование: точность, согласованность, дискриминативность	16
6.3. Устойчивость к распространению заданий	17
6.4. Подбор	18
6.5. NLP	19
6.6. Ограничения и план валидации на реальных данных	19
7. Интеграция с реестром ФСП	21
7.1. Схема взаимодействия	21
7.2. Протокол OpenID Connect	21
7.3. Данные реестра	22
7.4. Кандидат без истории ФСП	22
7.5. Переход на реальный ФСП ID и реестр	22
8. Описание API	24
8.1. Общие сведения	24
8.2. Примеры	24
8.3. Перечень эндпоинтов	25
9. Безопасность, персональные данные и защита от злоупотреблений	29
9.1. Персональные данные (152-ФЗ)	29
9.2. Аутентификация и авторизация	29
9.3. Защита кандидатов от недобросовестных работодателей	29
9.4. Интеграция с ATS работодателя	29
10. Используемые библиотеки и компоненты	31
11. Сборка, развёртывание и запуск	32
11.1. Запуск в Docker (рекомендуется)	32
11.2. Переменные окружения	32
11.3. Локальный запуск без Docker	32
11.4. Тесты и воспроизведение валидации	33
11.5. Развёртывание на сервере	33
12. План развития	33

1. Обзор решения

1.1. Задача и подход

Универсальные площадки сводят подбор к совпадению ключевых слов в самоописании: работодатель вручную разбирает сотни откликов, кандидат откликается вслепую. «ФСП Карьера» меняет направление инициативы. Кандидат один раз проходит обязательный путь — опрос по специализации, выбор предполагаемого грейда и адаптивный тест — и получает **катеорию** (специализация × грейд), подтверждённую тестом. Работодатель описывает потребность обычным текстом, система показывает подходящие категории с аналитикой рынка и ранжированных кандидатов с объяснением, а работодатель сам выходит на конкретного человека с предложением и **обязательной вилкой зарплаты**. Контакты кандидата раскрываются только после его согласия.

Пять решений, на которых держится продукт:

- **Категория — по тесту, а не по резюме.** Выдача формируется только из категорий, присвоенных тестированием. Самоописание влияет лишь на объяснения и текстовую близость.
- **Тест, устойчивый к утечкам.** Банк из 359 семейств заданий: у каждого кандидата свои варианты (данные, код на его языке, верный ответ вычисляется), адаптивный подбор заданий по модели IRT, контроль экспозиции и три детектора нечестного прохождения. Сложность вариантов одного семейства одинакова по построению, поэтому оценки разных кандидатов сопоставимы.
- **Объяснимое ранжирование.** Итоговая оценка складывается из пяти понятных компонент; каждая карточка кандидата содержит причины «за» и «против».
- **Прозрачный выход на контакт.** Приглашение содержит описание, вилку «от–до», компанию и способ связи; кандидат видит условия до общения, принимает или отклоняет с причиной.
- **ФСП ID и достижения.** Привязка профиля через OpenID Connect с путями Keycloak, достижения из реестра усиливают профиль; отсутствие истории ФСП не штрафует кандидата.

1.2. Роли и сквозной сценарий

Роль	Основные возможности
Кандидат	регистрация по e-mail с кодом; профиль вручную или из PDF-резюме с автораспознаванием; стандартизированный PDF-профиль; опрос и адаптивный тест; категория, грейд, история; приглашения с вилкой (принять, отклонить, пожаловаться); вакансии и отклики; регулярные задания работодателей; приватность, согласия 152-ФЗ, привязка ФСП ID
Работодатель	профиль компании; потребности и вакансии (NLP-разбор текста); подборки: рекомендованные категории, рынок, ранжирование с объяснением, фильтры поверх сохранённого результата; поиск по банку кандидатов; избранное; приглашения и их статусы; отклики на вакансии; регулярные задания и проверка решений; тест по вакансиям (как формируются задания)
Администратор	статистика банка заданий: экспозиция, дрейф решаемости, статус семейств; калибровка пилотных заданий
Система (фоновые правила)	предложение регулярных заданий раз в 7 дней; истечение приглашений через 14 дней; вывод «утёкших» заданий из ротации; уведомления в приложении и по e-mail

Сквозной сценарий демонстрации:

1. Кандидат регистрируется по e-mail, подтверждает адрес кодом и даёт согласие на обработку персональных данных.
2. Заполняет профиль или загружает PDF-резюме — ФИО, контакты, стек, стаж, роли и софт-скиллы распознаются автоматически.
3. Проходит опрос (специализация, основной язык, опыт, предполагаемый грейд) и адаптивный тест на выбранный грейд.
4. Получает категорию, например «Backend-разработчик · Middle», профиль по доменам и процентилю; при желании привязывает ФСП ID.

5. Работодатель описывает потребность текстом; система разбирает её, рекомендует категории и показывает ранжированных кандидатов с объяснениями.
6. Работодатель уточняет подборку фильтрами (исходный результат сохраняется) и отправляет приглашение с вилкой и способом связи.
7. Кандидат видит предложение с условиями и принимает его — контакты открываются обеим сторонам; или отклоняет с причиной.
8. Параллельно доступен классический сценарий: кандидат откликается на опубликованную вакансию, работодатель видит отклик с оценкой совпадения.

1.3. Соответствие требованиям ТЗ

Требование ТЗ	Как реализовано	Где посмотреть
Категоризация по обязательному пути: опрос, выбор грейда, тест	Опрос → выбор уровня → адаптивный тест; категория = специализация × грейд по результату теста	Кабинет кандидата → «Категория и грейд»; раздел 4
Тестирование, устойчивое к распространению заданий, с сопоставимой сложностью	Параметрические семейства заданий, уникальные варианты, IRT-шкала, контроль экспозиции, детекторы утечки	Разделы 4.2–4.6; страница «Методика»
Грейд не понижается принудительно; тест ниже/выше; ограничение частоты	Не подтвердил — сразу доступен тест уровнем ниже; подтвердил уверенно — уровнем выше; смена грейда не чаще раза в 90 дней, повтор уровня — через 30 дней	Раздел 4.5
Регулярные короткие задания от работодателей	Раз в 7 дней кандидату предлагается задание его категории; оценка работодателя влияет на силу профиля	Кабинеты → «Задания»; раздел 5.8
Выход на контакт по инициативе работодателя	Подборка → приглашение с вилкой, описанием, способом связи, без обязательной привязки к вакансии; статусы	Раздел 2.3; кабинет работодателя → «Приглашения»
Связь профиля с ФСП ID; совместимость с Keycloak	OpenID Connect (authorization code + PKCE), пути /realms/<realm>/protocol/openid-connect/*; реестр достижений	Раздел 7
Корректная обработка отсутствия истории ФСП	Сигнал ФСП = 0 без штрафа, явная пометка в объяснении, равный доступ к подбору	Разделы 5.5, 7.4
Профиль: форма или PDF с автораспознаванием; PDF-профиль	pdfminer.six + Natasha (NER) + онтология навыков; генерация PDF (ReportLab)	Кабинет кандидата → «Профиль и резюме»
Настройки: приватность, согласия, ФСП ID	Что видит работодатель, журнал согласий, отзыв, привязка ФСП ID	Кабинет кандидата → «Настройки и ФСП ID»
Ограничение доступа для кандидата и работодателя	Проверка роли и владельца на каждом эндпоинте; контакты — только после принятия приглашения или отклика	Разделы 2.4, 9
Профиль компании, потребность, подборка с обоснованием	NLP-разбор потребности, рекомендованные категории, причины «за/против» у каждого кандидата	Раздел 5
Поиск по банку с фильтрами: специализация, грейд, стек, ФСП	GET /employer/candidates и фильтры подборки	Кабинет работодателя → «Банк кандидатов»
Уточнение запроса без потери подборки	Подборка сохраняется; фильтры применяются поверх сохранённого результата	Раздел 5.7
Вилка «от-до» обязательна в вакансии и приглашении	Валидация схем API: обе границы обязательны, от ≤ до	Раздел 8
Статусы приглашений и откликов видны обеим сторонам	Отправлено → просмотрено → принято / отклонено / истекло / отозвано	Раздел 2.3
Регистрация по e-mail с подтверждением	Код из 6 цифр, хранится как HMAC, 30 минут, 5 попыток; письма через SMTP (в стенде — Mailpit)	Раздел 9.2
Вакансии и отклики (дополнительно)	Публикация, редактирование, закрытие; отклики со статусами и оценкой совпадения	Кабинеты обеих ролей

Требование ТЗ	Как реализовано	Где посмотреть
Защита от фиктивных вакансий и ATS (дополнительно)	Лимит приглашений, индекс доверия компании, жалобы, скрытие предложений ниже ожиданий; webhook в ATS	Разделы 9.3, 9.4
ML и NLP (преимущество)	IRT и адаптивное тестирование, NLP вакансий и резюме, TF-IDF-близость	Разделы 4, 5
OpenAPI, Docker, 152-ФЗ, адаптивная вёрстка	86 эндпоинтов в Swagger; docker compose up --build; согласия, экспорт и удаление данных; мобильное меню и адаптивные сетки	Разделы 8, 9, 11
Собственная процедура валидации и результаты	Monte-Carlo-симуляция с известной истиной, базовые линии, абляции	Раздел 6

2. Функциональная архитектура

2.1. Функциональные модули

Модуль	Функции	Пользователи
Регистрация и доступ	регистрация по e-mail с кодом подтверждения, вход по паролю или через ФСП ID, роли, JWT	все
Профиль кандидата	ручное заполнение, разбор PDF-резюме, PDF-профиль, приватность, согласия, выгрузка и удаление данных	кандидат
Опрос и тестирование	опрос по специализации, допуск к уровням с учётом ограничений, адаптивный тест, результат и история	кандидат
Категоризация	присвоение категории, история грейдов, профиль по доменам, статусы навыков по тесту	система
Подбор	разбор потребности (NLP), рекомендованные категории и рынок, ранжирование, объяснения, подборки и фильтры, поиск по банку, избранное	работодатель
Взаимодействие	приглашения, вакансии, отклики, жалобы, уведомления в приложении и по e-mail	обе роли
Регулярные задания	создание заданий, предложение кандидатам, решения, проверка и влияние на профиль	обе роли
Интеграция с ФСП	привязка и вход через ФСП ID, синхронизация достижений, показ работодателю	кандидат, система
Администрирование банка	экспозиция и дрейф заданий, вывод из ротации, калибровка пилотных заданий	администратор
Методика	публичная страница с описанием механик и результатами валидации	все

2.2. Путь кандидата

Регистрация → подтверждение e-mail → профиль (вручную или из PDF) → опрос → тест на выбранный грейд → решение по грейду. Если грейд подтверждён, кандидат получает категорию и попадает в банк кандидатов (при согласии на публикацию профиля и включённой видимости). Если не подтверждён, сразу доступен тест уровнем ниже; прежний грейд, если он был, сохраняется. После присвоения категории кандидат получает приглашения, может откликаться на вакансии и раз в неделю получает регулярное задание.

Кандидат без категории может откликаться на вакансии (работодатель видит пометку «без категории»), но не попадает в подборки — выдача строится только по категориям, присвоенным тестом.

2.3. Жизненный цикл приглашения и отклика

Статус приглашения	Когда наступает	Что видят стороны
Отправлено (sent)	работодатель отправил приглашение	кандидат — предложение с вилкой, компанией, способом связи и оценкой совпадения
Просмотрено (viewed)	кандидат впервые открыл приглашение	работодатель видит, что предложение прочитано
Принято (accepted)	кандидат принял	контакты открываются обеим сторонам; при настроенной интеграции уходит webhook в ATS
Отклонено (declined)	кандидат отклонил с причиной из справочника	работодатель видит причину (доход, стек, формат, компания, не ищу работу, другое)
Истекло (expired)	14 дней без ответа	приглашение закрыто
Отозвано (withdrawn)	работодатель отозвал	приглашение закрыто

Отклик на вакансию проходит статусы «отправлен → просмотрен → приглашение на интервью / отказ», кандидат может отозвать отклик. Отклик сам по себе открывает работодателю контакты кандидата: инициатива исходит от кандидата.

Правила отправки приглашения: вилка обязательна; у одной компании может быть только одно активное приглашение одному кандидату; не более 60 приглашений в сутки на компанию; если кандидат включил «не принимать предложения ниже моих ожиданий», приглашение с верхней границей вилки ниже ожиданий не отправляется (ответ 409 с пояснением).

2.4. Правила видимости данных

Данные кандидата	Работодатель до контакта	После принятия приглашения или отклика
ФИО	анонимный код вида C-37ABD2 (имя — только если кандидат разрешил)	видно
E-mail, телефон, Telegram, ссылки	скрыты	видны
Категория, проценты, профиль по доменам теста	видны	видны
Навыки и их статусы по тесту, опыт, образование	видны; названия прошлых работодателей — по настройке	видны
Ожидания по доходу	по настройке (по умолчанию видны)	видны
Достижения ФСП	по настройке (по умолчанию видны)	видны

Кандидат видит только собственные приглашения, отклики и задания. Работодатель видит только свои подборки, приглашения, вакансии и отклики на них. Каждый просмотр карточки кандидата и каждое раскрытие контактов записываются в журнал аудита.

3. Компонентная архитектура

3.1. Схема компонентов

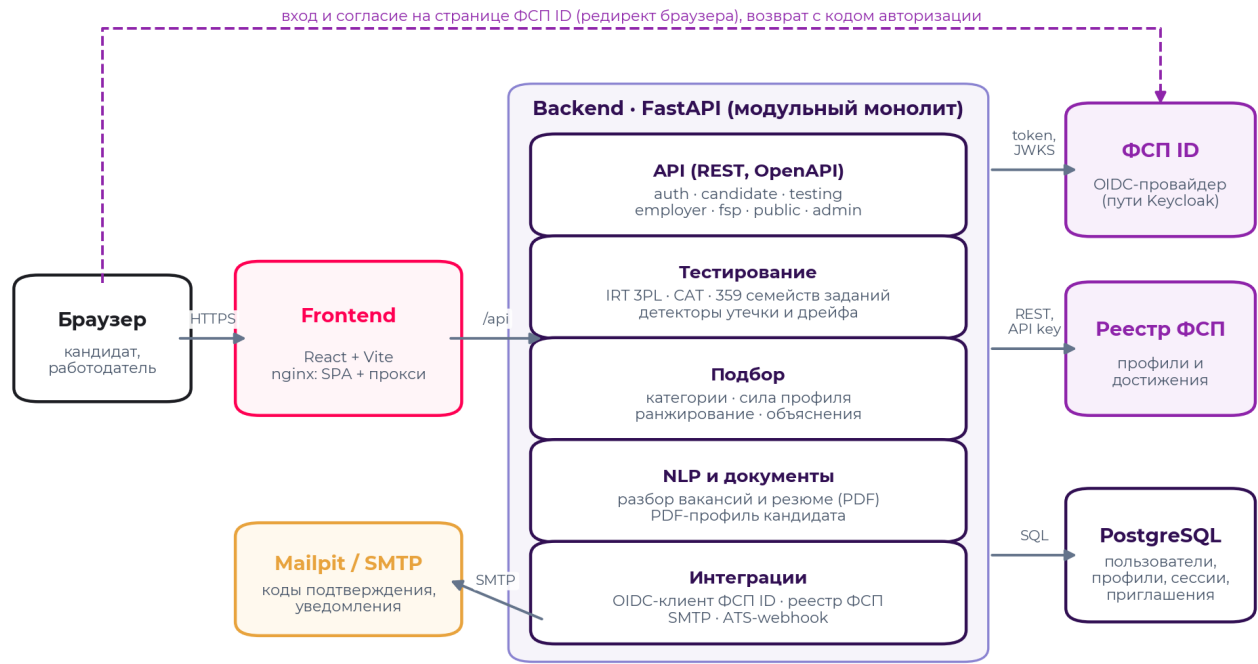


Рисунок 1. Компонентная архитектура: браузер, фронтенд на nginx, модульный монолит на FastAPI, PostgreSQL, ФСП ID, реестр ФСП, SMTP

Решение — модульный монолит с явным разделением слоёв, упакованный в Docker. Пять контейнеров: frontend (nginx отдаёт собранное SPA и проксирует /api на бэкэнд), backend (FastAPI), db (PostgreSQL 16), fsp-mock (OIDC-провайдер «ФСП ID» и реестр достижений с путями Keycloak), mailpit (SMTP-сервер для писем с кодами). Суммарный лимит памяти — около 2 ГБ.

3.2. Компоненты и ответственность

Компонент	Ответственность	Ключевые файлы
api	REST-роутеры, валидация входных данных (Pydantic), проверка ролей и владельца, коды ошибок	backend/app/api/*.py
services/testing	IRT 3PL, адаптивный алгоритм, банк заданий и генераторы вариантов, детекторы, превью теста по вакансии	irt.py, cat.py, bank/, integrity.py, service.py, preview.py
services/matching	сила профиля, статусы навыков, рекомендованные категории, ранжирование, объяснения, фильтры	profile.py, ranking.py
services/nlp	разбор вакансий (специализация, грейды, навыки, вилка, формат, город), разбор PDF-резюме	vacancy_parser.py, resume_parser.py
services/fsp	OIDC-клиент ФСП ID, загрузка участника из реестра, оценка достижений	client.py, scoring.py
services/pdf	стандартизированный PDF-профиль кандидата	profile_pdf.py
services(общие)	взаимодействия, регулярные задания, уведомления и аудит, представления кандидата	interactions.py, candidates.py, notify.py
models, core	модели SQLAlchemy, конфигурация, БД, безопасность (Argon2id, JWT, HMAC), почта	backend/app/models, backend/app/core
seed, validation	демо-данные и синтетическая популяция; процедура валидации и отчёты	backend/app/seed, backend/validation
Frontend	SPA для обеих полей: React 18, TypeScript, TanStack Query, Tailwind CSS, framer-motion; ленивые маршруты	frontend/src

Компонент	Ответственность	Ключевые файлы
ФСП ID (mock)	OIDC-провайдер (RS256, JWKS, PKCE) и реестр достижений по API-ключу	fsp-mock/app/main.py

Ядро тестирования (`irt.py`, `cat.py`, банк заданий) не зависит от базы данных и HTTP, поэтому одно и то же ядро работает в продукте и в симуляциях валидации — метрики раздела 6 получены на рабочем коде, а не на его модели.

3.3. Модель данных

Таблица	Назначение
users	учётные записи: e-mail, хеш пароля (Argon2id), роль, подтверждение e-mail
email_codes	коды подтверждения (HMAC), срок действия, число попыток
consents	журнал согласий: вид, дано или отозвано, время, устройство
notifications, audit_log	уведомления; журнал действий (просмотры карточек, раскрытие контактов, ФСП)
oauth_states	состояния OIDC: state, nonce, PKCE-verifier, назначение (привязка или вход), срок 10 минут
candidates	профиль: контакты, опыт, навыки, ожидания, категория (grade, grade_theta, grade_se), доменные оценки, статусы навыков, ФСП ID и достижения, сила профиля, приватность
survey_responses, grade_history	история опросов; история присвоения и смены грейда
test_sessions, test_responses	сессии теста (блюпринт, язык, θ , SE, результат); показанные варианты: семейство, seed варианта, ключ, ответ, время
item_stats	статистика семейств: параметры IRT, показы, наблюдаемая и ожидаемая решаемость, z дрейфа, статус (active, pretest, flagged, retired)
companies, vacancies	компании (индекс доверия, URL ATS-webhook); потребности и вакансии со структурированными полями
selections, shortlist	сохранённые подборки (потребность, категории, результат); избранное
invitations, applications	приглашения с вилкой и снимком объяснения; отклики
tasks, task_assignments, complaints	регулярные задания и решения; жалобы на приглашения

3.4. Производительность и масштабирование

Бэкенд не хранит состояние между запросами (сессии теста — в БД, токены — JWT), поэтому масштабируется репликами за балансировщиком. Подборка по пулу до 900 кандидатов ранжируется в среднем за 39 мс (p95 — 69 мс); полный запрос «текст потребности → подборка» в Docker-стенде на 700 кандидатах выполняется за 0,1–0,2 с. Тяжёлые объекты (модель классификации специализаций, NER-модель Natasha, индекс детектора «чужих вариантов») загружаются при старте в фоновом потоке. Фронтенд разделён на ленивые чанки по маршрутам: основной бандл — около 430 КБ.

4. Механика тестирования

4.1. Опрос и выбор уровня

Опрос фиксирует отрасль, специализацию (7 направлений: Backend, Frontend, Fullstack, Data Science / ML, аналитик данных, DevOps / SRE, QA), основной язык программирования (Python, Java, Go, JavaScript / TypeScript — в зависимости от специализации), опыт и предполагаемый грейд. Грейдов четыре: стажёр, Junior, Middle, Senior. Кандидат сам выбирает уровень теста; рекомендуется заявленный в опросе.

Специализация задаёт **блюпринт** — доли тестовых доменов. Например, для Backend: алгоритмы 14%, основной язык 22%, SQL 13%, базы данных 10%, HTTP и API 12%, архитектура 12%, безопасность 9%, Linux 4%, CI/CD 4%. Всего в банке 27 доменов.

4.2. Банк заданий: семейства и варианты

Единица банка — **семейство заданий**: шаблон с фиксированной темой, уровнем (1–5), типом ответа, ограничением времени и параметрами IRT. Для каждого показа генератор семейства строит **вариант** по seed, выведенному из токена сессии, семейства и номера показа: новые данные, массивы, таблицы, константы, имена; код — на основном языке кандидата. Верный ответ вычисляется самим генератором (для SQL-заданий запрос реально выполняется в SQLite на сгенерированных таблицах), варианты ответов перемешиваются, идентификаторы вариантов случайны.

Показатель банка	Значение
Семейств заданий	359 по 27 доменам
Параметрических (уникальные данные в каждом варианте)	125
Типы ответа	выбор одного варианта — 238, числовой ответ — 76, ввод ответа — 38, выбор нескольких — 7
Уровни сложности 1–5	38 / 93 / 121 / 78 / 29 семейств
Ограничение времени на задание	60–240 с (у большинства — 90 с)

Почему не «уникальное задание от нейросети каждому»: такая генерация не гарантирует сопоставимую сложность. В семействе меняются данные, но не структура рассуждения, поэтому трудность вариантов одинакова по построению и описывается одними параметрами IRT. Новые семейства — в том числе черновики, подготовленные с помощью LLM и прошедшие ревью эксперта, — входят в банк как пилотные: показываются без влияния на оценку и калибруются по реальным ответам (раздел 4.6).

4.3. Модель IRT и оценка уровня

Используется трёхпараметрическая логистическая модель:

$$P(\text{верно} \mid \theta) = c + (1 - c) / (1 + \exp(-a \cdot (\theta - b)))$$

где θ — уровень кандидата на общей шкале, a — дискриминативность задания, b — трудность, c — вероятность угадать (0,25 для выбора из четырёх вариантов, 0 для ввода ответа). Уровень оценивается методом EAP — апостериорное среднее на сетке $\theta \in [-4; 4]$ с одинаковым для всех кандидатов априорным распределением $N(0; 1,25^2)$. Из-за общей шкалы и общего prior результаты кандидатов, получивших разные задания, сопоставимы. Погрешность SE — апостериорное стандартное отклонение.

Грейдам соответствуют полосы шкалы θ с границами -1, 0 и 1: стажёр — ниже -1, Junior — [-1; 0), Middle — [0; 1), Senior — от 1. Процентиль кандидата — $\Phi(\theta)$ относительно рынка $N(0; 1)$.

4.4. Адаптивный алгоритм

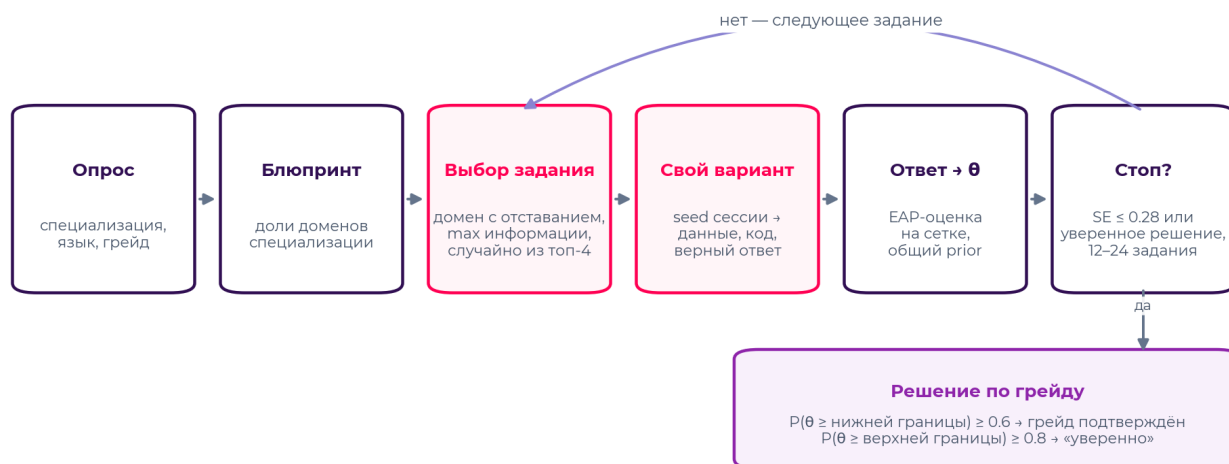


Рисунок 2. Цикл адаптивного теста: опрос, блюпринт, выбор задания, уникальный вариант, оценка θ , правило остановки, решение по грейду

- 1. Стартовая точка** — центр заявленного грейда; после первых трёх ответов точка выбора плавно смещается к текущей оценке θ (стартовая точка влияет только на выбор заданий, но не на оценку).
- 2. Домен** — тот, что сильнее всего отстаёт от своей доли в блюпринте.
- 3. Задание** — среди активных семейств домена, которые кандидат не видел за последние 180 дней и чья доля показов не превышает 30%, берутся четыре с максимальной информацией Фишера $I(\theta) = a^2 \cdot (P - c)^2 / (1 - c)^2 \cdot (1 - P) / P$, и одно из них выбирается случайно (randomesque).
- 4. Пилотные задания** — после четырёх рабочих заданий с вероятностью 12% показывается пилотное задание; в оценку оно не входит.
- 5. Остановка** — не раньше 12 и не позже 24 рабочих заданий; досрочно, если $SE \leq 0,28$ или решение по обеим границам грейда принято с уверенностью 0,92.

Параметр	Значение	Обоснование
Длина теста	12–24 задания, в среднем 20,4 (≈ 20 минут)	баланс точности и времени, подобран симуляцией
Порог остановки по SE	0,28	точность, достаточная для разделения соседних грейдов
Randomesque	случайный выбор из 4 лучших	разнообразие наборов заданий
Максимальная экспозиция семейства	30%	ограничение «популярных» заданий
Порог подтверждения грейда	$P(\theta \geq \text{нижней границы}) \geq 0,6$	ошибка «завысили грейд» дороже для работодателя, чем «предложили пересдать уровнем ниже»
Уверенное подтверждение	$P(\theta \geq \text{верхней границы}) \geq 0,8$	сразу открывается тест уровнем выше

4.5. Решение по грейду и правила пересдачи

По итогам теста выносится одно из трёх решений: «подтверждён», «подтверждён уверенно» (с предложением теста уровнем выше в течение 7 дней) или «пока не подтверждён». Правила, следующие из ТЗ:

- грейд не понижается принудительно: если тест на более высокий уровень не пройден, текущий грейд сохраняется;
- тест на уровень ниже текущего грейда недоступен — понижать себя не нужно; после неподтверждения доступен тест уровнем ниже заявленного;

- смена грейда — не чаще одного раза в 90 дней; повторная попытка того же уровня — через 30 дней; уверенно подтвердивший уровень может сразу пробовать следующий;
- при пересдаче кандидат получает другие семейства заданий (виденные за 180 дней исключаются), а внутри семейства — другой вариант;
- незавершённая сессия автоматически закрывается через 3 часа; время ответа контролируется сервером с запасом 15 секунд.

4.6. Обоснование устойчивости к распространению заданий

Угроза: задания и ответы расходятся между кандидатами («слитая база»). Защита построена в несколько слоёв, и каждый слой проверен экспериментом (раздел 6.3).

Угроза	Мера защиты	Эффект
Ответы из слитой базы	Параметрические варианты: данные и верный ответ уникальны для каждого показа	ответ чужого варианта неверен для своего
Списанные ответы	Детектор «чужого варианта»: ответ не подходит к своему варианту, но совпадает с ключом другого варианта того же семейства (вероятность случайного совпадения у отобранных семейств $\leq 2\%$)	при ≥ 2 таких ответах результат не меняет грейд до повторного теста под наблюдением
Заучивание популярных заданий	Адаптивный выбор, randomesque, ограничение экспозиции 30%, исключение виденного за 180 дней	у двух кандидатов одного уровня совпадает лишь 15% вопросов
Утечка статичных заданий	Мониторинг дрейфа: если задание систематически решают чаще, чем предсказывает модель ($z > 3$ при ≥ 30 показах), семейство выводится из ротации	обнаружено 67% утёкших семейств за 1800 прохождений при 0% ложных срабатываний
Подставное прохождение, подсказки	Person-fit $Iz < -2$ (несогласованный паттерн ответов) и ≥ 2 верных ответов на трудные задания быстрее 8% лимита времени	сессия помечается для проверки
Устаревание банка	Пилотные задания встраиваются без влияния на оценку и калибруются онлайн методом максимального правдоподобия при фиксированных θ (от 40 ответов)	банк пополняется без потери сопоставимости

Ключевой результат эксперимента: даже когда нечестный кандидат знает все задания 25 предыдущих кандидатов своей специализации, доля **незамеченного** завышения грейда (11,7%) остаётся на уровне честного прохождения (11,4% — это погрешность измерения), тогда как фиксированный тест в тех же условиях завышают 96,5% нечестных кандидатов.

4.7. Профиль по доменам и статусы навыков

По ответам каждого домена считается доменная оценка θ_d (EAP с prior в общей θ — при малом числе заданий оценка сжимается к общей). Навыки, которые домен теста измеряет напрямую (например, SQL, PostgreSQL — доменом «SQL», Docker — доменом «Docker и контейнеры»), получают статус по свидетельствам теста относительно уровня требований вакансии: «подтверждён тестом», «заявлен», «не подтвердился». Знание конкретных инструментов (Kafka, Spring, PyTorch) общий домен не проверяет — такие навыки остаются «заявленными». Основным языком из опроса считается заявленным навыком и проверяется тестом.

5. Механика категоризации и подбора

5.1. Категория кандидата

Категория — пара «специализация × грейд», где специализация выбрана в опросе, а грейд подтверждён тестом этой специализации. Категорию видит работодатель; именно по категориям строится выдача. Справочник: 7 специализаций × 4 грейда = 28 категорий; смежные специализации (например, Backend и Fullstack, ML и аналитика данных) учитываются при рекомендациях с понижающим коэффициентом.

5.2. Разбор потребности работодателя (NLP)

Работодатель описывает потребность обычным текстом. Разбор возвращает структурированный запрос, который можно поправить перед подбором:

- **специализация** — гибриды двух моделей: TF-IDF по символьным n-граммам (устойчив к русской морфологии) + логистическая регрессия, обученная на синтетическом корпусе из онтологии навыков и ролевых фраз; и правила по онтологии (вклад найденных навыков и заголовка). Итог — 0,4·модель + 0,6·правила, с уверенностью и альтернативами;
- **грейды** — по явным упоминаниям и опыту; **навыки** — онтология из 102 навыков с синонимами, разделение на обязательные и желательные («будет плюсом»);
- **вилка, формат работы, город** — правила и регулярные выражения.

Пример разбора текста «Ищем Python-разработчика уровня Middle в команду платежей. Обязательно: Python, PostgreSQL, Docker, REST API. Будет плюсом: Kafka. Зарплата 200 000 – 280 000 руб., гибриды, Москва.»: специализация Backend (уверенность 0,61), грейд Middle, обязательные навыки Python, PostgreSQL, Docker, REST API, желательный — Kafka, вилка 200–280 тыс. ₽, гибриды, Москва.

5.3. Рекомендованные категории и аналитика рынка

Для потребности система рекомендует основную категорию (запрошенные грейды), соседние грейды (ниже — с релевантностью 0,45: «растущий кандидат»; выше — 0,6: «сильнее, чем нужно, но дороже») и смежные специализации. По каждой категории показывается число кандидатов, сколько из них открыты к предложениям, средняя сила профиля, число кандидатов с достижениями ФСП, медиана ожиданий по доходу и доля кандидатов, чьи ожидания укладываются в вилку. Работодатель видит рынок до того, как начнёт приглашать.

5.4. Ранжирование

Пул кандидатов — только из рекомендованных категорий, присвоенных тестом. Оценка соответствия:

$$\text{score} = (0,30 \cdot \text{навыки} + 0,25 \cdot \text{сила профиля} + 0,20 \cdot \text{категория} + 0,15 \cdot \text{условия} + 0,10 \cdot \text{текст}) \times \text{доступность}$$

Компонента	Как считается
Навыки	среднее по обязательным навыкам: подтверждён тестом — 0,8, заявлен — 0,65, не подтвердился — 0,4, нет в заявленных, но домен уверенно выше уровня требований — 0,3, смежный навык — 0,3, нет данных — 0; итог 0,85·обязательные + 0,15·желательные. Тест отвечает на вопрос «владеет ли кандидат навыком на уровне требований» (нижняя граница запрошенного грейда), а не «насколько он силен» — уровень учитывают категория и сила профиля
Сила профиля	0,6·тест + 0,25·ФСП + 0,15·регулярные задания (раздел 5.5); тестовая часть — положение θ относительно полосы запрошенных грейдов
Категория	коэффициент специализации (1 — та же, 0,3–0,6 — смежная) × ($\frac{1}{2}$ ·совпадение грейда + $\frac{1}{2}$ ·вероятность, что θ кандидата лежит в запрошенных грейдах с учётом погрешности теста)
Условия	0,5·доход + 0,3·формат + 0,2·город: ожидания в вилке — 1, до +15% — 0,5, выше — 0,15; формат и город — совпадение, удалённость, готовность к переезду
Текст	TF-IDF-близость (символьные 3–5-граммы) текста потребности и текстов профиля

Компонента	Как считается
Доступность	множитель за фактическую недоступность: ожидания заметно выше вилки $\times 0,6$, немного выше $\times 0,9$; другой город без готовности к переезду $\times 0,5$; несовместимый формат $\times 0,75$. Такие кандидаты не исключаются, а опускаются ниже

Если работодатель отметил, что важен опыт ФСП, кандидаты без привязанного ФСП ID получают мягкое понижение $\times 0,85$, а не исключение. Внутри одной категории порядок определяется силой подтверждённого профиля — выше те, у кого лучше результаты теста и есть достижения ФСП, как требует ТЗ.

5.5. Сила профиля и достижения ФСП

сила профиля = $0,60 \cdot \text{тест} + 0,25 \cdot \text{ФСП} + 0,15 \cdot \text{активность}$

- **Тест** — положение θ в полосе грейда: $0,15 + 0,85 \cdot (\theta - \text{нижняя граница}) / (\text{ширина полосы})$, в пределах $[0; 1]$. Самоописание в силу профиля не входит.
- **ФСП** — сигналы достижений агрегируются «шумным ИЛИ»: $1 - P(1 - 0,85 \cdot s_i)$, плюс бонус за спортивный разряд (МС $+0,15$, КМС $+0,10$, I разряд $+0,05$), не больше 1. Сигнал достижения $s_i = \text{уровень} \times \text{место} \times \text{релевантность дисциплины специализации} \times \text{свежесть}$: уровень — региональный 0,35, межрегиональный 0,5, всероссийский 0,8, международный 1,0; место — 1-е 1,0, 2-е 0,85, 3-е 0,7, финалист (топ-10%) 0,55, участие 0,25; релевантность — матрица «дисциплина ФСП $\times$ специализация» (например, продуктивное программирование для Backend — 1,0, алгоритмическое — 0,9); свежесть — период полураспада 3 года. Несколько средних результатов дают больше одного, но счёт насыщается.
- **Активность** — решённые регулярные задания: объём $(1 - e^{-(n/3)}) \times \text{качество по оценкам работодателей} \times \text{затухание с периодом полураспада 60 дней}$.
- **Нет истории ФСП** — сигнал равен 0, это отсутствие бонуса, а не штраф: кандидат остаётся в выдаче на общих основаниях, а в объяснении прямо написано «Нет истории ФСП — на попадание в подборку не влияет».

5.6. Объяснения и вероятность отклика

Каждая карточка кандидата в подборке содержит процент соответствия, вклад компонент и причины «за» и «против», например: «Категория „Backend-разработчик · Middle“ подтверждена тестом — результат выше, чем у 62% кандидатов», «Подтверждены тестом на уровне требований: PostgreSQL, REST API», «Заявлены, но тест показал уровень ниже требуемого: ...», «ФСП: призёр всероссийского чемпионата...», «Ожидания по доходу немного выше вилки (до 15%)», «Другой город и не готов к переезду».

Перед отправкой приглашения работодатель видит оценку вероятности отклика (высокая / средняя / низкая) — логистическая функция от соответствия вилки ожиданиям, формата, города, давности активности и открытости к предложениям — и может пересчитать её для другой вилки.

5.7. Подборки, фильтры и поиск по банку

Подборка сохраняется целиком: потребность, рекомендованные категории, ранжированный результат со срезом атрибутов. Фильтры (специализация, грейды, навыки — включая режим «только подтверждённые тестом», наличие достижений ФСП, формат, город с учётом релокации, потолок ожиданий, минимальный процент соответствия) применяются поверх сохранённого результата без пересчёта и без потери исходной подборки; пересчитать подборку на актуальной базе можно отдельной кнопкой. Поиск по всему банку кандидатов доступен отдельно с фильтрами по специализации, грейду, категории, стеку и наличию достижений ФСП.

5.8. Регулярные задания работодателей

Работодатель создаёт короткие задания для категории (специализация и грейды): описание, критерии и, при желании, эталонное решение. Раз в 7 дней кандидату с присвоенной категорией, открытому к предложениям, предлагается одно новое задание его профиля со сроком 7 дней; можно решить, предложить подход или пропустить. Решение получает предварительную автооценку близости к эталону, работодатель выставляет оценку 1–5 с комментарием. Оценки и давность решений входят в силу профиля (компонента активности), а в объяснении появляется «Решено заданий работодателей: N (профиль актуален)».

6. Процедура валидации и результаты

6.1. Принцип

Эталонной разметки на хакатоне нет, поэтому качество проверяется на **синтетической популяции с известной истиной** — стандартный подход для адаптивных тестов (Monte-Carlo-симуляция CAT). У каждого синтетического кандидата заданы скрытые истинные специализация, уровень θ , доменные способности, навыки, ожидания и география; ответы на задания генерируются моделью IRT по истинным способностям. 35% кандидатов «завышают» резюме: заявляют грейд на ступень выше и добавляют модные навыки. Системы видят только наблюдаемые данные: резюме, ответы на тест, ФСП, условия. Метрики считаются против скрытой истины.

Все эксперименты используют рабочий код продукта (ядро теста, банк заданий, ранжирование, NLP) и воспроизводятся одной командой: `python -m validation.run_all` (около 2 минут). Отчёты: `backend/validation/reports/*.json` и `SUMMARY.md`; те же результаты показаны на странице «Методика».

6.2. Тестирование: точность, согласованность, дискриминативность

Популяция — 1500 кандидатов.

Метрика	Значение
Корреляция оценки θ с истинным θ / RMSE	0,919 / 0,43
Средняя длина теста	20,4 задания (10–90-й перцентили: 12–24)
Точность решения «грейд подтверждён»	82,2% (ложные подтверждения 8,7%, ложные отказы 23,0%)
Итоговый грейд совпал с истинным	68,1% (самооценка в резюме — 56,6%)
Грейд в пределах ± 1 ступени	95,9%
Завышение грейда	2,4% (самооценка в резюме — 36,1%)
Повторное прохождение (другие задания): тот же грейд	68,2%; взвешенная $\kappa = 0,796$; $r(\theta_1, \theta_2) = 0,817$
Дискриминативность: медиана индекса D / доля заданий с $D \geq 0,3$	0,432 / 81,9% ($D < 0,2$ — 3,3%)
Медиана точечно-бисериальной корреляции	0,365 (параметрические семейства — $D = 0,568$, статичные — 0,383)
Сдвиг оценки по языкам кода (Backend)	Python $-0,19$, Java $-0,15$, Go $-0,17$, JavaScript $-0,15$ — разброс 0,04 при RMSE 0,41–0,47

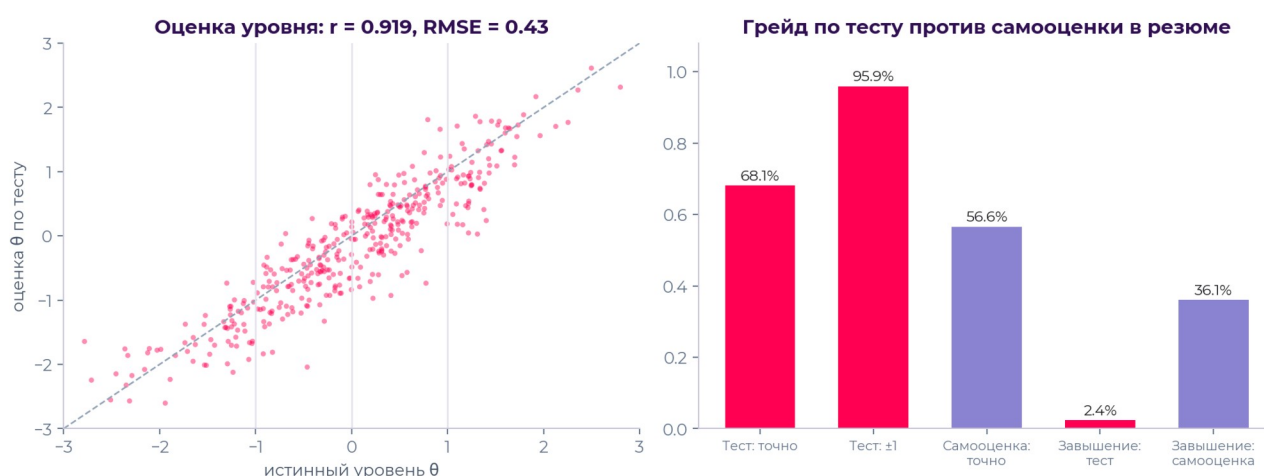


Рисунок 3. Оценка уровня против истинного и сравнение грейда по тесту с самооценкой

Асимметрия ошибок решения намеренная: порог 0,6 делает ложное подтверждение (8,7%) реже ложного отказа (23,0%). Отказ не понижает грейд и не закрывает путь — кандидат сразу может пройти тест уровнем ниже, — а ложное подтверждение обходится работодателю дороже. Матрица «истинный грейд → присвоенный» показывает, что ошибки сосредоточены у соседних границ и смещены вниз:

Истинный \ присвоенный	Стажёр	Junior	Middle	Senior	Нет категории
Стажёр	156	12	0	0	57
Junior	121	370	14	0	4
Middle	0	168	323	10	0
Senior	0	0	92	173	0

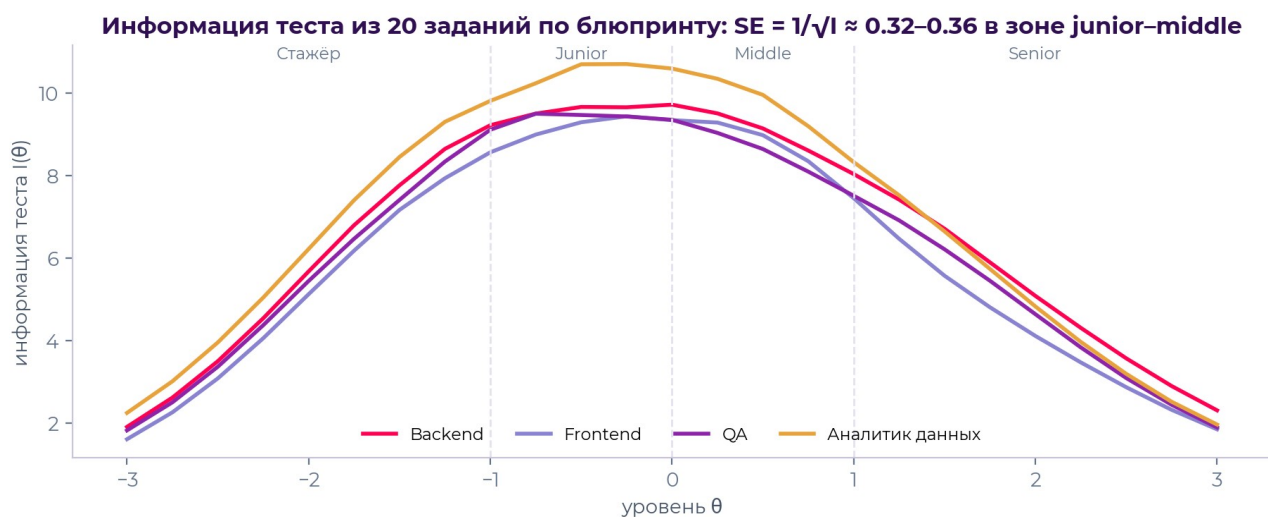


Рисунок 4. Информационная функция теста по специализациям

6.3. Устойчивость к распространению заданий

Экспозиция и пересечение. Используется 94,7% семейств банка; максимальная доля показов одного семейства — 28,9% (95-й перцентиль — 17,5%). У двух кандидатов одного уровня и специализации (худший случай) совпадает 49,3% семейств, но лишь 15,4% вопросов в идентичной форме.

Атака «слитой базой». 315 нечестных кандидатов заявляют грейд на ступень выше и знают все задания (с ответами), показанные К предыдущим кандидатам их специализации. Сравниваются три системы: фиксированный тест из 20 статичных заданий, адаптивный тест без параметрических вариантов и наша система.

Система	Честно (K = 0)	K = 5	K = 25	K = 100
Фиксированный тест	24,8%	96,5%	96,5%	96,5%
Адаптивный тест без вариантов	12,4%	93,0%	95,9%	96,5%
ФСП Карьера: завышение	11,4%	53,3%	58,7%	61,0%
ФСП Карьера: незамеченное завышение	11,4%	8,9%	11,7%	16,5%
ФСП Карьера: доля помеченных сессий	1,0%	80,0%	81,0%	77,8%

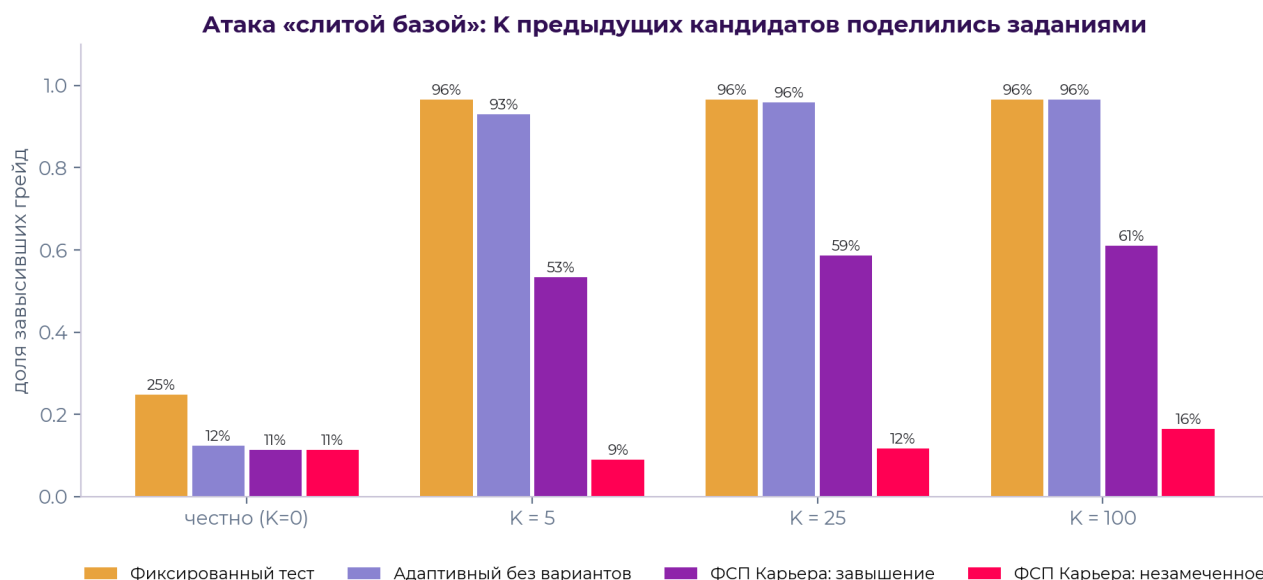


Рисунок 5. Атака «слитой базой»: доля зависивших грейд

Списывание из базы срабатывает только на статичных заданиях; на параметрических оно оставляет «чужие» ответы, и детектор пометает 78–81% таких сессий при 1% ложных пометок у честных. Помеченный результат не меняет грейд до повторного теста под наблюдением, поэтому **незамеченное** завышение остаётся на уровне честного прохождения.

Обнаружение утечки статичных заданий. Сценарий: в Backend / Python утекли 12 самых популярных статичных семейств, их знают 20% кандидатов. Доля выявленных утёкших семейств (TPR) при пороге $z > 3$: после 300 прохождений — 25%, 900 — 42%, 1500 — 50%, 1800 — 67%; ложных срабатываний (FPR) — 0% во всех точках. Выявленное семейство заменяется, результаты кандидатов не аннулируются.

Ошибки калибровки. Если априорные параметры заданий неточны ($b \pm 0,4$, $a \times e^{N(0; 0,25)}$), точность грейда снижается умеренно: 73,2% против 75,7% у «оракула» с истинными параметрами. Онлайн-калибровка по рабочим ответам снижает ошибку трудности с 0,396 до 0,259 и возвращает точность грейда до 74,0%.

6.4. Подбор

900 кандидатов, 60 потребностей разных специализаций и грейдов. Релевантность пары «потребность — кандидат» (метка 0–3) задаётся скрытой истиной: специализация, грейд, владение обязательными навыками, ожидания по доходу и география; релевантен — метка ≥ 2 (в среднем 27 релевантных кандидатов на потребность). Сравниваются поиск по ключевым словам (TF-IDF по резюме, как на универсальных площадках), классические фильтры по самоописанию (специализация, заявленный грейд, зарплата, город, ранжирование по покрытию заявленных навыков) и наша система, а также её абляции.

Система	P@10	nDCG @10	MRR	Нерелевантных в топ-10	«Зависивших себя» в топ-10
Поиск по ключевым словам	0,272 ± 0,046	0,293	0,437	42,2%	8,7%
Фильтры по самоописанию	0,743 ± 0,054	0,801	0,919	1,2%	17,7%
ФСП Карьера	0,845 ± 0,049	0,839	0,983	3,2%	2,5%
ФСП Карьера, потребность из текста (NLP)	0,843 ± 0,049	0,838	0,983	3,5%	2,3%
Абляция: категория из самоописания	0,728 ± 0,058	0,771	0,875	4,0%	16,8%
Абляция: без ФСП	0,842 ± 0,049	0,839	0,983	3,2%	3,0%
Абляция: без проверки навыков тестом	0,852 ± 0,049	0,859	0,983	2,7%	2,2%

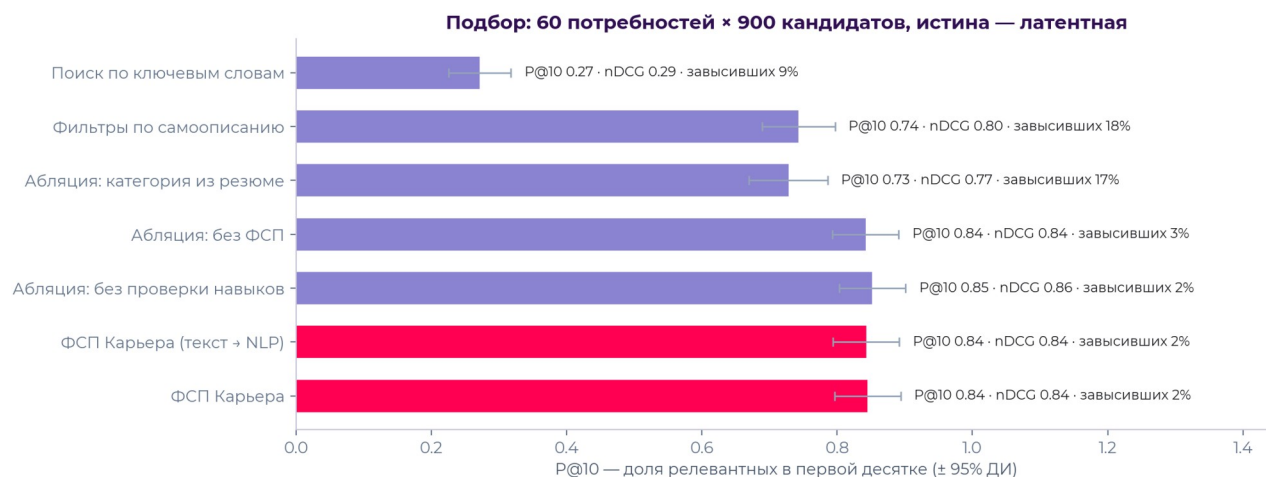


Рисунок 6. Доля релевантных кандидатов в первой десятке по системам

Выводы:

- категория по тесту — главный источник качества: замена её на самописание снижает P@10 с 0,845 до 0,728 и в семь раз увеличивает долю «зависивших себя» в топ-10 (16,8% против 2,5%);
- сквозной сценарий «текст вакансии → NLP → подборка» теряет лишь 0,002 P@10 по сравнению со структурированным запросом;
- вклад ФСП в синтетику мал (+0,003): достижения есть у меньшинства кандидатов и коррелируют с уровнем, который уже измерен тестом; на реальных данных вклад ФСП — предмет пилота;
- проверка навыков тестом не меняет P@10 значимо (0,845 против 0,852, в пределах доверительного интервала): в синтетике резюме честно перечисляют 92% навыков. Её ценность — в объяснимости («подтверждён тестом / не подтвердился») и в защите от навыков, заявленных для вида. В ходе валидации отвергнут вариант с непрерывной шкалой владения навыком: он повторно учитывал уровень θ и поднимал переqualificированных кандидатов (P@10 0,828);
- среднее время ранжирования пула до 900 кандидатов — 39 мс (p95 — 69 мс).

6.5. NLP

Задача	Набор	Результат
Специализация вакансии	28 текстов, размеченных командой	100% (только модель — 96,4%, только правила — 100%)
Грейды / вилка / формат / город вакансии	те же 28 текстов	100% / 100% / 100% / 92,9%
Навыки вакансии	те же 28 текстов	точность 0,953, полнота 0,977, F1 0,961; желательные навыки распознаны в 87,5%
Резюме из PDF: ФИО / e-mail / телефон / Telegram / город	60 синтетических PDF в форматах hh.ru и свободном	95% / 100% / 100% / 100% / 98,3%
Резюме: стаж ±0,5 года / навыки	те же 60 PDF	98,3% (средняя ошибка 0,05 года) / F1 0,954

6.6. Ограничения и план валидации на реальных данных

- Эталон синтетический: проверяются свойства процедур при известной истине. Выводы зависят от допущений генератора — они явно описаны в коде (backend/app/seed/synthetic.py) и могут быть оспорены и изменены.
- Тексты вакансий для проверки NLP написаны командой; перед запуском нужна проверка на выборке реальных вакансий.
- План пилота с ФСП: 1) 100–200 участников проходят тест, эксперты ФСП независимо оценивают грейд по интервью — согласие теста с экспертами (κ) и калибровка порогов; 2) повторное прохождение через 2–4 недели — устойчивость грейда; 3) онлайн-калибровка параметров

заданий на реальных ответах; 4) работодатели-партнёры размечают релевантность топ-20 подборок — $P@10$ и $nDCG$ на реальных парах; 5) A/B-сравнение конверсии приглашений в принятие с базовыми фильтрами.

7. Интеграция с реестром ФСП

7.1. Схема взаимодействия

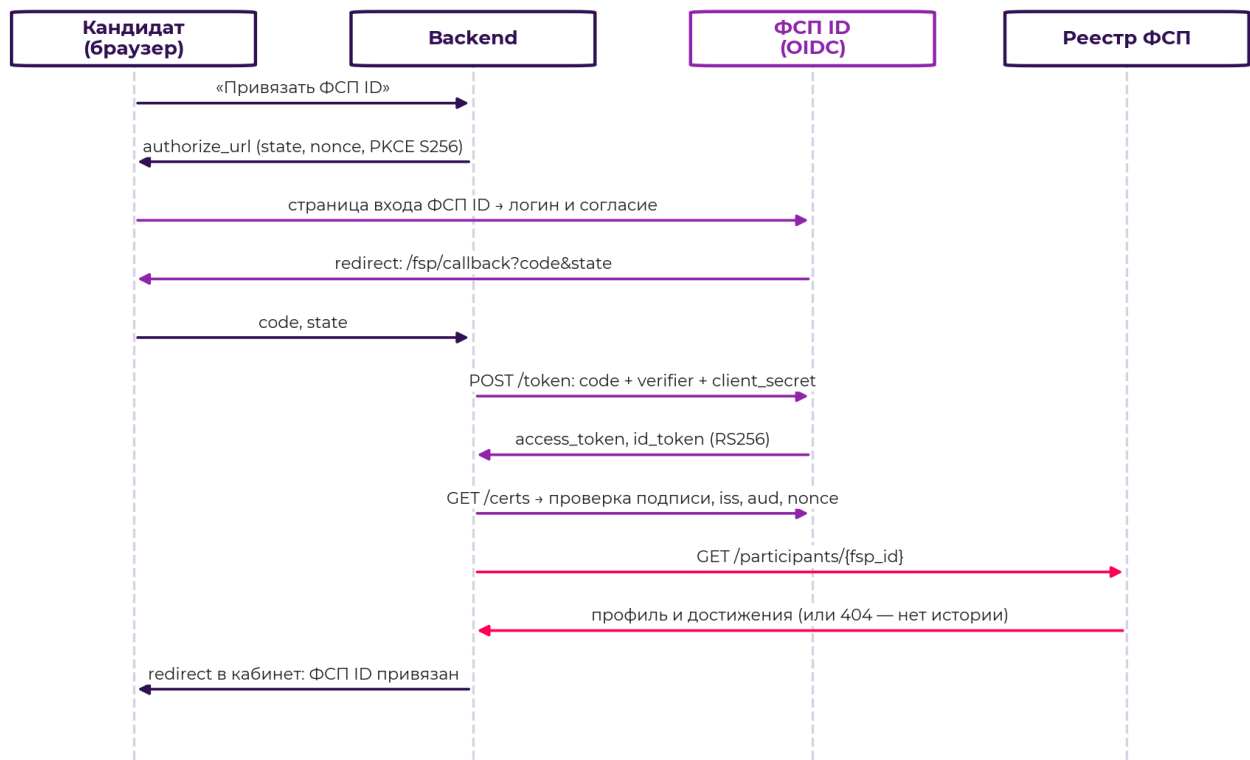


Рисунок 7. Последовательность привязки ФСП ID: OIDC authorization code + PKCE, проверка id_token, загрузка участника из реестра

Кандидат нажимает «Привязать ФСП ID» (или «Войти через ФСП ID» на странице входа). Бэкенд создаёт состояние авторизации (state, nonce, PKCE-verifier, срок 10 минут) и перенаправляет браузер на страницу входа ФСП ID. После входа и согласия провайдер возвращает код авторизации на /api/v1/fsp/callback. Бэкенд обменивает код на токены (с PKCE-verifier и секретом клиента), проверяет подпись id_token по ключам JWKS, издателя, аудиторию и nonce, извлекает ФСП ID и запрашивает участника в реестре. Достижения сохраняются в профиле, сила профиля пересчитывается.

7.2. Протокол OpenID Connect

Пути эндпоинтов совпадают с Keycloak, поэтому подключение к реальному ФСП ID на базе Keycloak сводится к смене адреса издателя и учётных данных клиента.

Назначение	Эндпоинт провайдера
Метаданные	GET /realms/fsp/.well-known/openid-configuration
Авторизация	GET /realms/fsp/protocol/openid-connect/auth (response_type=code, scope=openid profile email, code_challenge_method=S256)
Токены	POST /realms/fsp/protocol/openid-connect/token (grant_type=authorization_code, code_verifier)
Ключи подписи	GET /realms/fsp/protocol/openid-connect/certs (JWKS, RS256)
Профиль	GET /realms/fsp/protocol/openid-connect/userinfo

Меры безопасности: PKCE S256, одноразовый state с ограниченным сроком, проверка nonce, проверка подписи и iss/aud/exp у id_token, один ФСП ID — один профиль (повторная привязка к

другому профилю отклоняется), секрет клиента хранится только на бэкенде. Идентификатор участника берётся из клейма `fsp_id`, при его отсутствии — из `preferred_username` или `sub`.

Вход через ФСП ID: если профиль с этим ФСП ID уже есть — выполняется вход; если нет — создаётся учётная запись кандидата с e-mail, подтверждённым провайдером, и фиксируется согласие на обработку данных.

7.3. Данные реестра

Реестр отдаёт участника по ФСП ID: `GET /registry/api/v1/participants/{fsp_id}` (сервисный ключ X-API-Key или access-токен самого участника) и поиск `GET /registry/api/v1/participants?q=` (только сервисный ключ). Набор данных выбран так, чтобы работодатель мог проверить каждое достижение.

Поле	Описание
<code>fsp_id</code> , <code>full_name</code> , <code>email</code> , <code>region</code> , <code>birth_year</code>	идентификация участника
<code>sport_rank</code>	спортивный разряд (МС, КМС, I разряд)
<code>rating.points</code> , <code>rating.season</code>	рейтинг участника в сезоне
<code>achievements[].event_id</code> , <code>event</code> , <code>date</code>	соревнование и дата
<code>achievements[].discipline</code>	дисциплина: продуктивное, алгоритмическое программирование, системы ИБ, беспилотные авиационные системы, робототехника
<code>achievements[].level</code>	уровень: региональный, межрегиональный, всероссийский, международный
<code>achievements[].place</code> , <code>participants</code>	место и число участников (для расчёта «финалист, топ-10%»)
<code>achievements[].team</code> , <code>role</code>	команда и роль в команде (капитан, участник)
<code>achievements[].result_url</code> , <code>verified</code>	ссылка на протокол результатов; признак подтверждения ФСП

Работодателю достижения показываются компактно: заголовок («Призёр всероссийского чемпионата...»), лучший результат с местом и годом, список с дисциплинами и ссылками на протоколы. Кандидат может скрыть достижения настройками приватности.

7.4. Кандидат без истории ФСП

Возможны три случая, все обрабатываются явно: ФСП ID не привязан; привязан, но в реестре нет участника (404) — профиль привязан с пустым списком достижений; участник есть, но без достижений. Во всех случаях сигнал ФСП равен 0: кандидат попадает в подборки на общих основаниях, а в объяснении написано «Нет истории ФСП — на попадание в подборку не влияет, бонус не начислен» или «ФСП ID привязан, достижений в реестре пока нет». Абляция подтверждает, что отсутствие ФСП-компоненты почти не меняет качество выдачи (раздел 6.4), то есть система не дискриминирует кандидатов без соревновательного опыта.

7.5. Переход на реальный ФСП ID и реестр

1. Зарегистрировать в Keycloak ФСП клиента `fsp-career` (confidential, Standard Flow, PKCE S256), redirect URI `https://<домен>/api/v1/fsp/callback`.
2. Добавить в токен клейм `fsp_id` (mapper атрибута пользователя) — либо использовать `preferred_username`.
3. Указать переменные окружения `FSP_PUBLIC_ISSUER=https://<keycloak>/realms/<realm>`, `FSP_CLIENT_SECRET`, `FSP_REGISTRY_URL`, `FSP_REGISTRY_API_KEY`; mock-сервис из состава стенда исключить.
4. Согласовать с ФСП контракт реестра (раздел 7.3) или адаптер к существующему API — изменения локализованы в `backend/app/services/fsp/client.py`.
5. Синхронизация: при привязке, по кнопке «Обновить достижения» и, в развитии, по событиям реестра (webhook о новых протоколах).

8. Описание API

8.1. Общие сведения

- Базовый путь — `/api/v1`; формат — JSON (UTF-8); загрузка резюме — `multipart/form-data` (PDF до 5 МБ).
- Спецификация OpenAPI генерируется автоматически: Swagger UI — `/docs`, ReDoc — `/redoc`, JSON — `/openapi.json`. Для каждого эндпоинта описаны параметры, схемы запросов и ответов, коды ошибок и примеры.
- Аутентификация — `Authorization: Bearer <JWT>`; токен выдаётся при входе по паролю или через ФСП ID, срок действия — 12 часов. Кнопка `Authorize` в Swagger поддерживает OAuth2 password flow.
- Ошибки — `{"detail": "текст"}`; ошибки валидации — код 422 с перечнем полей. Основные коды: 400 — некорректный запрос, 401 — нет или истёк токен, 403 — недостаточно прав, 404 — не найдено или чужой объект, 409 — конфликт состояния (например, активное приглашение уже есть), 413 — файл слишком большой, 429 — превышен лимит попыток или приглашений.

8.2. Примеры

Вход и получение токена:

```
POST /api/v1/auth/login
{"email": "employer@demo.ru", "password": "demo12345"}

200 OK
{"access_token": "eyJhbGciOiJIUzI1NiIsInR5cGU": "bearer",
"user": {"id": 324, "email": "employer@demo.ru", "role": "employer",
"email_verified": true}}
```

Разбор потребности:

```
POST /api/v1/employer/vacancies/parse
{"text": "Ищем Python-разработчика уровня Middle в команду платежей. Обязательно:
Python, PostgreSQL, Docker, REST API. Будет плюсом: Kafka.
Зарплата 200 000 - 280 000 руб., гибрид, Москва."}

200 OK
{"specialization": "backend", "specialization_confidence": 0.608, "grades": ["middle"],
"must_skills": ["python", "postgresql", "docker", "rest"], "nice_skills": ["kafka"],
"salary_from": 200000, "salary_to": 280000, "work_format": "hybrid", "city": "Москва", ...}
```

Подборка (ответ сокращён):

```
POST /api/v1/employer/selections
{"text": "<текст потребности>", "title": "Python-разработчик (платежи)"}

201 Created
{"id": 12, "need": {...}, "total": 78,
"categories": [{"specialization": "backend", "grade": "middle", "primary": true,
"count": 18, "with_fsp": 3, "median_salary": 250000,
"share_in_budget": 0.72, ...}, ...],
"results": [{"public_id": "C-346E6B", "match": 51, "grade": "middle",
"components": {"skills": 0.33, "strength": 0.28, "category": 0.93,
"conditions": 0.94, "text": 0.14},
"reasons": [{"kind": "plus",
"text": "Категория «Backend-разработчик · Middle» подтверждена тестом..."}, ...],
"likelihood": {"p": 0.88, "label": "высокая"}}, ...]}
```

Приглашение (вилка обязательна; привязка к вакансии — нет):

```
POST /api/v1/employer/invitations
{"candidate_id": 57, "title": "Backend-разработчик в платёжную команду",
"message": "Развиваем платёжный шлюз на Python и PostgreSQL...",
"salary_from": 220000, "salary_to": 280000, "work_format": "hybrid",
"contact_method": "Telegram @techpulse_hr"}

201 Created – приглашение со статусом "sent";
409 – у кандидата уже есть ваше активное приглашение / вилка ниже его ожиданий;
429 – достигнут суточный лимит приглашений
```

Ответ кандидата: POST /api/v1/candidate/invitations/{id}/accept открывает контакты обеим сторонам; POST /api/v1/candidate/invitations/{id}/decline с телом {"reason": "salary", "comment": "..."} закрывает приглашение с причиной.

8.3. Перечень эндпоинтов

Все пути указаны относительно /api/v1. Полная спецификация с схемами — в Swagger (/docs); таблица сгенерирована из неё скриптом docs/gen_api_table.py.

Группа	Метод	Путь	Назначение	Коды ответа	JWT
Аутентификация	POST	/auth/login	Вход по e-mail и паролю	200, 401, 403, 422, 429	нет
Аутентификация	GET	/auth/me	Текущий пользователь	200	да
Аутентификация	POST	/auth/register	Регистрация кандидата или работодателя	201, 409, 422	нет
Аутентификация	POST	/auth/resent-code	Повторная отправка кода	200, 422	нет
Аутентификация	POST	/auth/token	OAuth2 password flow (для кнопки Authorize в Swagger)	200, 422	нет
Аутентификация	POST	/auth/verify-email	Подтверждение e-mail кодом	200, 400, 404, 422, 429	нет
Справочники	GET	/reference	Все справочники одним запросом	200	нет
Справочники	GET	/reference/skills	Онтология навыков	200	нет
Кандидат: профиль	DELETE	/candidate/account	Удалить аккаунт и персональные данные	200	да
Кандидат: профиль	GET	/candidate/consents	Журнал согласий	200	да
Кандидат: профиль	POST	/candidate/consents	Дать или отозвать согласие (152-ФЗ)	200, 422	да
Кандидат: профиль	GET	/candidate/dashboard	Сводка для главной страницы кандидата	200	да
Кандидат: профиль	GET	/candidate/export	Выгрузка всех моих данных (152-ФЗ)	200	да
Кандидат: профиль	PUT	/candidate/privacy	Настройки приватности: что видит работодатель	200, 422	да
Кандидат: профиль	GET	/candidate/profile	Мой профиль	200	да
Кандидат: профиль	PUT	/candidate/profile	Сохранить профиль (ручное заполнение или подтверждение распознанного резюме)	200, 422	да
Кандидат: профиль	GET	/candidate/profile/pdf	Стандартизированный PDF-профиль	200	да
Кандидат: профиль	POST	/candidate/resume	Загрузить PDF-резюме и распознать поля	200, 400, 413, 422	да
Кандидат: опрос и тестирование	GET	/testing/eligibility	Какие уровни теста доступны сейчас (с учётом ограничений по частоте)	200	да
Кандидат: опрос и тестирование	GET	/testing/history	История тестов и смен грейда	200	да
Кандидат: опрос и тестирование	POST	/testing/sessions	Начать тест на выбранный грейд (или продолжить незавершённый)	200, 409, 422	да
Кандидат: опрос и тестирование	GET	/testing/sessions/{token}	Состояние сессии: текущее задание или результат	200, 422	да
Кандидат: опрос и тестирование	POST	/testing/sessions/{token}/abandon	Прервать тест	200, 422	да

Группа	Метод	Путь	Назначение	Коды ответа	JWT
Кандидат: опрос и тестирование	POST	/testing/sessions/{token}/answer	Ответить на текущее задание	200, 409, 422	да
Кандидат: опрос и тестирование	GET	/testing/survey	Вопросы опроса по отрасли и специализации	200	нет
Кандидат: опрос и тестирование	POST	/testing/survey	Сохранить ответы опроса	200, 422	да
Кандидат: опрос и тестирование	GET	/testing/survey/history	История ответов на опрос	200	да
Кандидат: приглашения, вакансии, задания	GET	/candidate/applications	Мои отклики и их статусы	200	да
Кандидат: приглашения, вакансии, задания	POST	/candidate/applications/{app_id}/withdraw	Отозвать отклик	200, 422	да
Кандидат: приглашения, вакансии, задания	GET	/candidate/invitations	Мои приглашения от работодателей	200, 422	да
Кандидат: приглашения, вакансии, задания	GET	/candidate/invitations/{inv_id}	Открыть приглашение (статус → «просмотрено»)	200, 422	да
Кандидат: приглашения, вакансии, задания	POST	/candidate/invitations/{inv_id}/accept	Принять приглашение — контакты раскрываются	200, 409, 422	да
Кандидат: приглашения, вакансии, задания	POST	/candidate/invitations/{inv_id}/complain	Пожаловаться на приглашение (фиктивная вакансия и т. п.)	200, 422	да
Кандидат: приглашения, вакансии, задания	POST	/candidate/invitations/{inv_id}/decline	Отклонить приглашение с причиной	200, 409, 422	да
Кандидат: приглашения, вакансии, задания	GET	/candidate/tasks	Регулярные задания от работодателей (система периодически предлагает новые)	200	да
Кандидат: приглашения, вакансии, задания	POST	/candidate/tasks/{ta_id}/skip	Пропустить задание	200, 422	да
Кандидат: приглашения, вакансии, задания	POST	/candidate/tasks/{ta_id}/submit	Отправить решение или подход	200, 409, 422	да
Кандидат: приглашения, вакансии, задания	GET	/notifications	Мои уведомления	200, 422	да
Кандидат: приглашения, вакансии, задания	POST	/notifications/read	Отметить все уведомления прочитанными	200	да
Кандидат: приглашения, вакансии, задания	GET	/vacancies	Опубликованные вакансии (с оценкой совпадения для кандидата)	200, 422	да
Кандидат: приглашения, вакансии, задания	GET	/vacancies/{vac_id}	Вакансия	200, 422	да
Кандидат: приглашения, вакансии, задания	POST	/vacancies/{vac_id}/apply	Откликнуться на вакансию	200, 409, 422	да
Работодатель	PUT	/employer/applications/{app_id}	Изменить статус отклика	200, 422	да
Работодатель	GET	/employer/candidates	Поиск по банку кандидатов	200, 422	да
Работодатель	GET	/employer/candidates/{cid}	Карточка кандидата (контакты — только после принятия приглашения/отклика)	200, 422	да
Работодатель	GET	/employer/candidates/{cid}/fsp	Достижения ФСП кандидата (если он разрешил показ)	200, 422	да
Работодатель	GET	/employer/candidates/{cid}/pdf	Стандартизированный PDF-профиль кандидата	200, 422	да
Работодатель	GET	/employer/company	Профиль компании	200	да

Группа	Метод	Путь	Назначение	Коды ответа	JWT
Работодатель	PUT	/employer/company	Обновить профиль компании	200, 422	да
Работодатель	GET	/employer/dashboard	Сводка работодателя	200	да
Работодатель	GET	/employer/invitations	Отправленные приглашения и их статусы	200, 422	да
Работодатель	POST	/employer/invitations	Пригласить кандидата (вилка 3П обязательна)	201, 409, 422, 429	да
Работодатель	POST	/employer/invitations/{inv_id}/withdraw	Отозвать приглашение	200, 422	да
Работодатель	GET	/employer/likelihood/{cid}	Оценка вероятности принятия приглашения при заданной вилке	200, 422	да
Работодатель	GET	/employer/selections	История подборок	200	да
Работодатель	POST	/employer/selections	Сформировать подборку по потребности	201, 422	да
Работодатель	GET	/employer/selections/{sid}	Подборка с уточняющими фильтрами (поверх сохранённого результата)	200, 422	да
Работодатель	POST	/employer/selections/{sid}/refresh	Пересчитать подборку на текущей базе кандидатов	200, 422	да
Работодатель	GET	/employer/shortlist	Избранные кандидаты	200	да
Работодатель	POST	/employer/shortlist	Добавить кандидата в избранное	201, 422	да
Работодатель	DELETE	/employer/shortlist/{candidate_id}	Убрать из избранного	200, 422	да
Работодатель	POST	/employer/submissions/{ta_id}/review	Оценить решение (влияет на актуальность профиля кандидата)	200, 422	да
Работодатель	GET	/employer/tasks	Мои регулярные задания	200	да
Работодатель	POST	/employer/tasks	Создать короткое задание для кандидатов категории	201, 422	да
Работодатель	GET	/employer/tasks/submissions	Решения кандидатов на проверку	200, 422	да
Работодатель	PUT	/employer/tasks/{tid}	Изменить задание	200, 422	да
Работодатель	GET	/employer/vacancies	Мои потребности и вакансии	200	да
Работодатель	POST	/employer/vacancies	Создать потребность (приватно) или вакансию (опубликовать)	201, 422	да
Работодатель	POST	/employer/vacancies/parse	Разобрать текст потребности (NLP): специализация, грейд, навыки, вилка	200, 422	нет
Работодатель	GET	/employer/vacancies/{vid}	Потребность/вакансия	200, 422	да
Работодатель	PUT	/employer/vacancies/{vid}	Редактировать потребность/вакансию	200, 422	да
Работодатель	GET	/employer/vacancies/{vid}/applications	Отклики на вакансию (с оценкой соответствия)	200, 422	да
Работодатель	POST	/employer/vacancies/{vid}/close	Закрыть вакансию	200, 422	да
Работодатель	GET	/employer/vacancies/{vid}/test-preview	Как по этой вакансии формируются задания теста	200, 422	да
Работодатель	GET	/employer/views	Кого из кандидатов просматривали (аудит)	200	да
Интеграция с ФСП	GET	/fsp/callback	Redirect URI для OIDC-провайдера ФСП ID	200, 422	нет
Интеграция с ФСП	DELETE	/fsp/link	Отвязать ФСП ID	200	да
Интеграция с ФСП	GET	/fsp/link	Начать привязку ФСП ID (возвращает URL авторизации ФСП)	200	да
Интеграция с ФСП	GET	/fsp/login	Начать вход через ФСП ID	200	нет
Интеграция с ФСП	POST	/fsp/sync	Обновить достижения из реестра ФСП	200, 409	да
Публичное: методика и оценка	POST	/eval/rank	Стенд оценки: ранжирование переданных кандидатов под текст вакансии	200, 422	нет
Публичное: методика и оценка	GET	/public/methodology	Результаты валидации механик тестирования и подбора	200	нет
Публичное: методика и оценка	GET	/public/stats	Статистика платформы	200	нет
Администрирование	GET	/admin/items	Статистика банка заданий: экспозиция и дрейф	200	да

Группа	Метод	Путь	Назначение	Коды ответа	JWT
Администрирование	POST	/admin/items/calibrate	Откалибровать пилотные задания	200	да
Службное	GET	/health	Проверка работоспособности	200	нет

9. Безопасность, персональные данные и защита от злоупотреблений

9.1. Персональные данные (152-ФЗ)

- Согласие на обработку персональных данных — обязательное условие регистрации; согласие на публикацию профиля работодателям — отдельное. Каждое согласие и его отзыв фиксируются в журнале с датой и устройством; отзыв согласия на публикацию скрывает профиль из банка кандидатов.
- Принцип минимизации: работодатель до контакта видит анонимный код и профессиональные данные; контакты — только после принятия приглашения или собственного отклика кандидата.
- Кандидат может выгрузить все свои данные (`GET /candidate/export`) и удалить аккаунт с персональными данными (`DELETE /candidate/account`).
- Все просмотры карточек и раскрытия контактов журналируются; кандидат видит число просмотров своего профиля, работодатель — историю своих просмотров.
- Для продуктива: размещение в РФ, шифрование дисков и резервных копий, регламент хранения журналов, уведомление Роскомнадзора об обработке ПДн.

9.2. Аутентификация и авторизация

- Пароли — Argon2id; коды подтверждения e-mail — 6 цифр, хранятся только как HMAC-SHA256, действуют 30 минут, не более 5 попыток ввода.
- JWT HS256 со сроком 12 часов; секрет задаётся переменной окружения.
- Ограничение частоты: вход — 8 попыток за 5 минут на e-mail, повторная отправка кода — 3 за 2 минуты (в продуктиве — общий лимитер на Redis).
- Разграничение прав по ролям (кандидат, работодатель, администратор) на уровне зависимостей FastAPI; проверка владельца объекта на каждом запросе — чужой объект неотличим от несуществующего (404).
- Бэкенд в контейнере работает от непривилегированного пользователя; CORS ограничен адресами фронтенда; секреты — только в переменных окружения.

9.3. Защита кандидатов от недобросовестных работодателей

- Суточный лимит приглашений на компанию (60) и запрет повторного приглашения при активном — защита от массовых рассылок.
- Индекс доверия компании: растёт при подтверждении домена (e-mail сотрудника совпадает с доменом сайта компании), снижается при жалобах кандидатов; показывается кандидату вместе с приглашением.
- Жалоба на приглашение (фиктивная вакансия, спам, несоответствие вилки, некорректное общение) — с понижением индекса доверия и основанием для модерации.
- Кандидат может не принимать предложения с вилкой ниже своих ожиданий; вилка всегда видна до начала общения.
- Развитие: модерация вакансий с высоким риском (аномальная вилка, массовые приглашения), верификация компании по ИНН через открытые реестры, блокировка при повторных жалобах.

9.4. Интеграция с ATS работодателя

В профиле компании задаётся URL webhook; при принятии приглашения и при отклике на вакансию платформа отправляет событие (`invitation.accepted`, `application.created`) с контактами кандидата — в обоих случаях они уже раскрыты компании по правилам видимости. Концепция развития: подпись событий HMAC и повторные попытки с очередью, двусторонняя синхронизация статусов (интервью,

оффер, найм) через REST API, готовые коннекторы к распространённым ATS (Huntflow, Talantix, «Поток»), выгрузка стандартизированного PDF-профиля в карточку кандидата ATS.

10. Используемые библиотеки и компоненты

Все компоненты — открытое ПО под разрешительными лицензиями. Точные версии зафиксированы в `backend/requirements.txt`, `fsp-mock/requirements.txt` и `frontend/package-lock.json`.

Компонент	Версия	Назначение	Лицензия
Python	3.11	среда бэкенда	PSF
FastAPI	0.142.2	REST API, OpenAPI	MIT
Starlette	1.7.0	ASGI-фреймворк	BSD-3
Uvicorn	0.54.0	ASGI-сервер	BSD-3
Pydantic / pydantic-settings	2.13.5 / 2.15.0	схемы и конфигурация	MIT
email-validator	2.3.0	проверка e-mail	Unlicense
python-multipart	0.0.32	загрузка файлов	Apache-2.0
SQLAlchemy	2.1.3	ORM	MIT
psycopg (binary)	3.3.6	драйвер PostgreSQL	LGPL-3.0
PyJWT / cryptography	2.15.1 / 50.0.2	JWT, проверка RS256	MIT / Apache-2.0, BSD
argon2-cffi	25.1.0	хеширование паролей	MIT
httpx	0.28.1	HTTP-клиент (ФСП ID, реестр, ATS)	BSD-3
NumPy / SciPy	2.4.6 / 1.17.1	ИРТ, вычисления	BSD-3
scikit-learn	1.9.1	TF-IDF, логистическая регрессия	BSD-3
Natasha	1.6.0	NER для разбора резюме	MIT
pdfminer.six	20260107	извлечение текста из PDF	MIT
ReportLab	5.0.1	генерация PDF-профиля	BSD
pytest	9.1.1	тесты	MIT
Jinja2	3.1.6	страница входа ФСП ID (mock)	BSD-3
React / React DOM	18.3.1	интерфейс	MIT
React Router	6.30.6	маршрутизация	MIT
TanStack Query	5.104.1	работа с API и кэш	MIT
Tailwind CSS	3.4.19	стили	MIT
framer-motion	11.18.2	анимации	MIT
Recharts	2.15.4	графики	MIT
lucide-react	1.52.0	иконки	ISC
react-markdown / remark-gfm	9.1.0 / 4.0.1	отображение текстов заданий	MIT
prism-react-renderer	2.4.1	подсветка кода в заданиях	MIT
clsx	2.1.1	классы	MIT
Шрифт Montserrat (@fontsource)	5.3.0	фирменная типографика	OFL-1.1
TypeScript / Vite	5.9.3 / 5.4.21	сборка фронтенда	Apache-2.0 / MIT
PostCSS / Autoprefixer	8.5.28 / 10.6.1	обработка CSS	MIT
PostgreSQL	16 (alpine)	СУБД	PostgreSQL License
nginx	1.27 (alpine)	раздача SPA, обратный прокси	BSD-2
Node.js	22 (alpine)	сборка фронтенда в Docker	MIT
Mailpit	latest	SMTP-сервер стенда	MIT

11. Сборка, развёртывание и запуск

11.1. Запуск в Docker (рекомендуется)

Требования: Docker Desktop или Docker Engine с Compose v2, свободные порты 8080, 8000, 8090, 8025, около 2 ГБ оперативной памяти.

1. Получите код: `git clone <адрес репозитория>` и перейдите в папку проекта.
2. При необходимости скопируйте `.env.example` в `.env` и измените значения (по умолчанию всё работает без него).
3. Соберите и запустите стенд: `docker compose up --build`. Первая сборка занимает 5–8 минут; при первом старте бэкенд создаёт схему БД и наполняет её демо-данными (700 кандидатов, компании, вакансии — 20–30 секунд).
4. Откройте `http://localhost:8080` — стенд готов, когда открывается главная страница.
5. Остановка — `docker compose down` (данные сохраняются в томе `pgdata`); полный сброс с повторным наполнением — `docker compose down -v`.

Адрес	Назначение
<code>http://localhost:8080</code>	приложение
<code>http://localhost:8080/docs</code>	Swagger UI (OpenAPI)
<code>http://localhost:8080/methodology</code>	методика и результаты валидации
<code>http://localhost:8025</code>	Mailpit — письма с кодами подтверждения
<code>http://localhost:8090</code>	ФСП ID (mock): OIDC-провайдер и реестр

Демо-аккаунты (пароль `demo12345`, на странице входа выбираются одним кликом): `employer@demo.ru` — работодатель, `candidate@demo.ru` — кандидат с категорией Backend · Middle, `newbie@demo.ru` — новый кандидат для сквозного сценария, `admin@demo.ru` — администратор. Учётные записи ФСП ID: `alice` (призёр, три достижения) и `gleb` (нет истории), пароль `fsp12345`.

11.2. Переменные окружения

Переменная	По умолчанию	Назначение
<code>PUBLIC_URL</code>	<code>http://localhost:8080</code>	публичный адрес приложения (редиректы OIDC, ссылки в письмах)
<code>FSP_PUBLIC_ISSUER</code>	<code>http://localhost:8090/realms/fsp</code>	издатель ФСП ID; для Keycloak — <code>https://<host>/realms/<realm></code>
<code>SECRET_KEY</code>	демо-значение	секрет подписи JWT и HMAC кодов — обязательно сменить
<code>POSTGRES_PASSWORD</code>	демо-значение	пароль БД
<code>FSP_CLIENT_SECRET</code> , <code>FSP_REGISTRY_API_KEY</code>	демо-значения	учётные данные клиента ФСП ID и ключ реестра
<code>EMAIL_DEV_MODE</code>	<code>true</code>	код подтверждения дублируется в ответе API и интерфейсе (только для демо)
<code>SEED_DEMO</code> , <code>SEED_CANDIDATES</code>	<code>true</code> , 700	наполнение пустой БД демо-данными

11.3. Локальный запуск без Docker

Требования: Python 3.11, Node.js 20+.

1. Бэкенд (по умолчанию SQLite в `backend/data`): `python -m venv .venv`, активируйте окружение, `pip install -r backend/requirements.txt`, затем в папке `backend` — `uvicorn app.main:app --port 8000`.
2. Mock ФСП ID в отдельном терминале: `pip install -r fsp-mock/requirements.txt`, затем в папке `fsp-mock` — `uvicorn app.main:app --port 8090`.

3. Фронтенд в отдельном терминале: в папке frontend — `npm ci` и `npm run dev`; приложение — `http://localhost:5173`, запросы `/api` проксируются на порт 8000.

11.4. Тесты и воспроизведение валидации

- `cd backend && python -m pytest -q` — 491 тест: банк заданий (корректность всех генераторов и ключей), сквозной сценарий кандидата и работодателя, правила видимости, PDF, NLP.
- `cd backend && python -m validation.run_all` — полный прогон валидации (около 2 минут), отчёты в `backend/validation/reports`.
- `python docs/figures/make_figures.py` — иллюстрации документации по свежим отчётам; `python docs/gen_api_table.py` — таблица эндпоинтов из работающего стенда.

11.5. Развёртывание на сервере

Для виртуального сервера (2 vCPU, 4 ГБ RAM) достаточно того же `docker-compose.yml`: задайте `PUBLIC_URL` и `FSP_PUBLIC_ISSUER` с публичными адресами, смените секреты, отключите `EMAIL_DEV_MODE` и укажите реальный SMTP, поставьте перед frontend TLS-терминатор (Caddy, nginx или балансировщик облака). Для масштабирования бэкенд запускается несколькими репликами — состояние хранится в PostgreSQL.

12. План развития

1. **Пилот с ФСП** (1–2 месяца): подключение к ФСП ID на Keycloak и реестру, экспертная валидация грейдов, онлайн-калибровка банка на реальных ответах, разметка релевантности работодателями-партнёрами.
2. **Банк заданий**: рост до 800+ семейств, задания с исполнением кода в песочнице, конвейер «черновик LLM → ревью эксперта → пилотное задание → калибровка».
3. **Подбор**: обучение весов ранжирования на реальных исходах (принятие приглашений, найм), learning-to-rank поверх объяснимых компонент, рекомендации кандидату «каких навыков не хватает до следующего грейда».
4. **Доверие и безопасность**: верификация компаний, модерация вакансий, прокторинг повторного теста под наблюдением.
5. **Интеграции**: коннекторы ATS, события реестра ФСП, мобильное приложение.